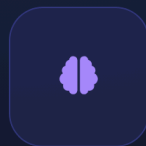


Sprint 3

Agentic AI Python Tutor — Neue Features



AGENT-GEDÄCHTNIS



CODE-REVIEW CHAIN



LERNPLAN-GENERATOR

AGENDA



SLIDE 3 — JOURNEY

Lisa's Lernstunde

Eine Studentin vom Login bis zum personalisierten Lernplan — alle neuen Features in einer echten Session.



SLIDE 4 — GEDÄCHTNIS

Kein Neustart mehr

Der Agent erinnert sich session-übergreifend — *"Letzte Woche haben wir Loops besprochen..."* ohne dass Lisa es wiederholen muss.



SLIDE 5 — REVIEW

3-Ebenen-Feedback

Syntax ● · Stil ● · Architektur ● — jede Ebene separat, damit Lisa weiß, *was zuerst zu fixen ist.*



SLIDE 6 — LERNPLAN

Personalisiert

Plan basiert auf echten Skill-Scores — schwache Themen zuerst, *nicht eine generische To-do-Liste.*



SLIDE 7 — AUSBLICK

Nächster Sprint

Adaptiver Hinweis-Dialog · Fehlermeldungen auf Deutsch · Code-Umschreibungs-Tool — Sprint 4 Planung.



SLIDE 8 — DEMO

Live Demo

System in Aktion — alle neuen Sprint-3-Features live zeigen.

USER JOURNEY

Lisa's Lernstunde — Eine Session von Anfang bis Ende



Lisa — Python-Anfängerin, 2. Semester. Sie hat letzte Woche Loops gelernt und will heute ihren ersten Code reviewen lassen.

**Dashboard**

Login → zwei klare
Optionen: *KI Tutor*
oder *Python Kurs*

`DashboardView.tsx`**Tutor startet**

"Hallo! 🖐️ *Ich bin dein
KI-Tutor!*" — Roboter
begrüßt sie mit
Animation

`TutorTransition.tsx`**Tutor erinnert sich**

Agent: "*Letzte Woche
haben wir Loops
besprochen — weiter
damit?*"

`memory_service.py`**Code Review**

Ihr Loop-Code
bekommt **3 Ebenen**
Feedback: Syntax 🔴,
Stil 🟡, Architektur 🔵

`RunnableSequence`**Lernplan**

Personalisierter
Wochenplan:
*schwache Skills
zuerst*, max. 3 Skills
pro Woche

`/learning-plan`**Nächste Woche**

Lisa kommt zurück —
Tutor weiß noch alles.
Kein Neustart.

`memory_summary in DB`

WIE FUNKTIONIERT'S? — AGENT-GEDÄCHTNIS

memory_service.py — Eigene Memory-Implementierung

⚡ LLM-AUFRUF 1 — ANTWORT MIT KONTEXT

Wenn Lisa eine Nachricht schickt, holt `load_memory()` ihren Summary aus der Datenbank und packt ihn als Kontext in den System-Prompt. Dann antwortet `run_chat()` — mit dem Wissen über vergangene Sessions.

⚡ LLM-AUFRUF 2 — MEMORY AKTUALISIEREN

Nach der Antwort ruft `update_memory()` das LLM ein zweites Mal auf: alter Summary plus neue Nachrichten rein — neuer, kompakter Summary raus. Der wird direkt in der DB gespeichert.

Wichtig: Wir speichern *nicht* den rohen Chat-Verlauf — nur den Summary. Das skaliert viel besser, und jeder User hat seinen eigenen Eintrag in der DB. Pro Nachricht also **zwei LLM-Aufrufe** — einer für die Antwort, einer für das Gedächtnis-Update.

WIE FUNKTIONIERT'S? — CODE-REVIEW CHAIN

Code-Review Chain — 3 Schritte · 3 LLM-Aufrufe

Lisa schickt ihren Code rein, und dann läuft eine `RunnableSequence` durch — **drei Schritte, drei separate LLM-Aufrufe.**

● SCHRITT 1 — SYNTAX

Das LLM sucht nur nach echten Fehlern — `SyntaxError`, `NameError`, mit Zeilennummer.

● SCHRITT 2 — STIL

Anderer Prompt, anderes LLM — schaut nur auf PEP8, Naming, Lesbarkeit.

● SCHRITT 3 — BEST PRACTICES

Architektur, Effizienz, ob der Code pythonisch ist.

Der Output von Schritt 1 geht als Input in Schritt 2 — **echte Chain**, nicht nur drei separate Aufrufe hintereinander. Im Frontend sieht Lisa dann drei aufklappbare Bereiche — **rot** für Fehler, **gelb** für Stil, **blau** für Best Practices.

WIE FUNKTIONIERT'S? — LERNPLAN-GENERATOR

Personalisierter Lernplan — Basierend auf echten Schwächen

Lisa fragt nach einem Lernplan — und kriegt keinen generischen Wochenplan, sondern einen der auf ihren **echten Skill-Scores** basiert. Schwache Themen kommen zuerst, max. drei Skills pro Woche.

Der Endpunkt `POST /learning-plan` liest die gespeicherten Scores aus der DB und übergibt sie dem LLM — das LLM entscheidet dann, in welcher Reihenfolge und welchem Tempo Lisa die Themen angehen soll.

Der Plan ist kein statisches Dokument — er ändert sich, wenn Lisa neue Themen bearbeitet und ihre Scores sich verbessern. Nächste Woche sieht der Plan automatisch anders aus.

NÄCHSTER SPRINT

Was kommt als nächstes?



Adaptiver Hinweis-Dialog

Der Tutor erkennt, wann ein Lernender nicht weiterkommt, und bietet kontextbezogene Hinweise in drei Stufen an — ohne die Lösung direkt zu verraten.

☰ Stufe 1 · Stufe 2 · Lösung

Hint-on-Demand · progressiv · LLM-generiert

SPRINT 4



Fehler-Erklärung auf Deutsch

Python-Fehlermeldungen werden automatisch erkannt, übersetzt und auf Deutsch verständlich erklärt — ideal für deutschsprachige Lernende.

🔍 Error-Detection · DE-Übersetzung · Beispiel

Traceback-Parsing · LLM-Erklärung · kontextsensitiv

SPRINT 4



Code-Umschreibungs-Tool

Eingabe: funktionierender aber unschöner Code. Ausgabe: sauberer, pythonischer Code mit Erklärung der Verbesserungen — als Lernmaterial.

🔄 Diff-Ansicht · PEP8 · Idiomatic Python

vorher / nachher · inline Kommentare · LLM-refactor

SPRINT 4

Live Demo